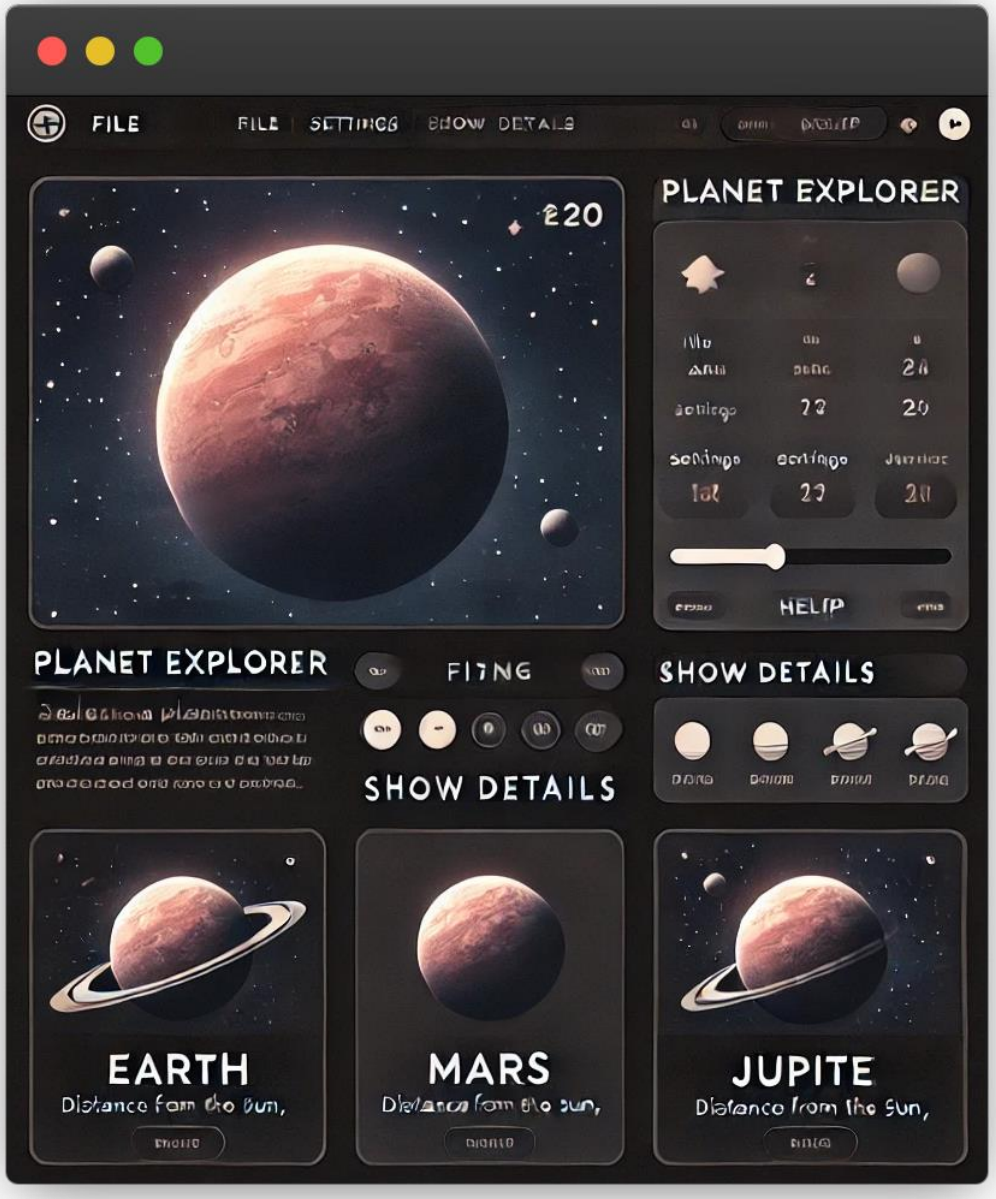
Chapter 14 15 16 31

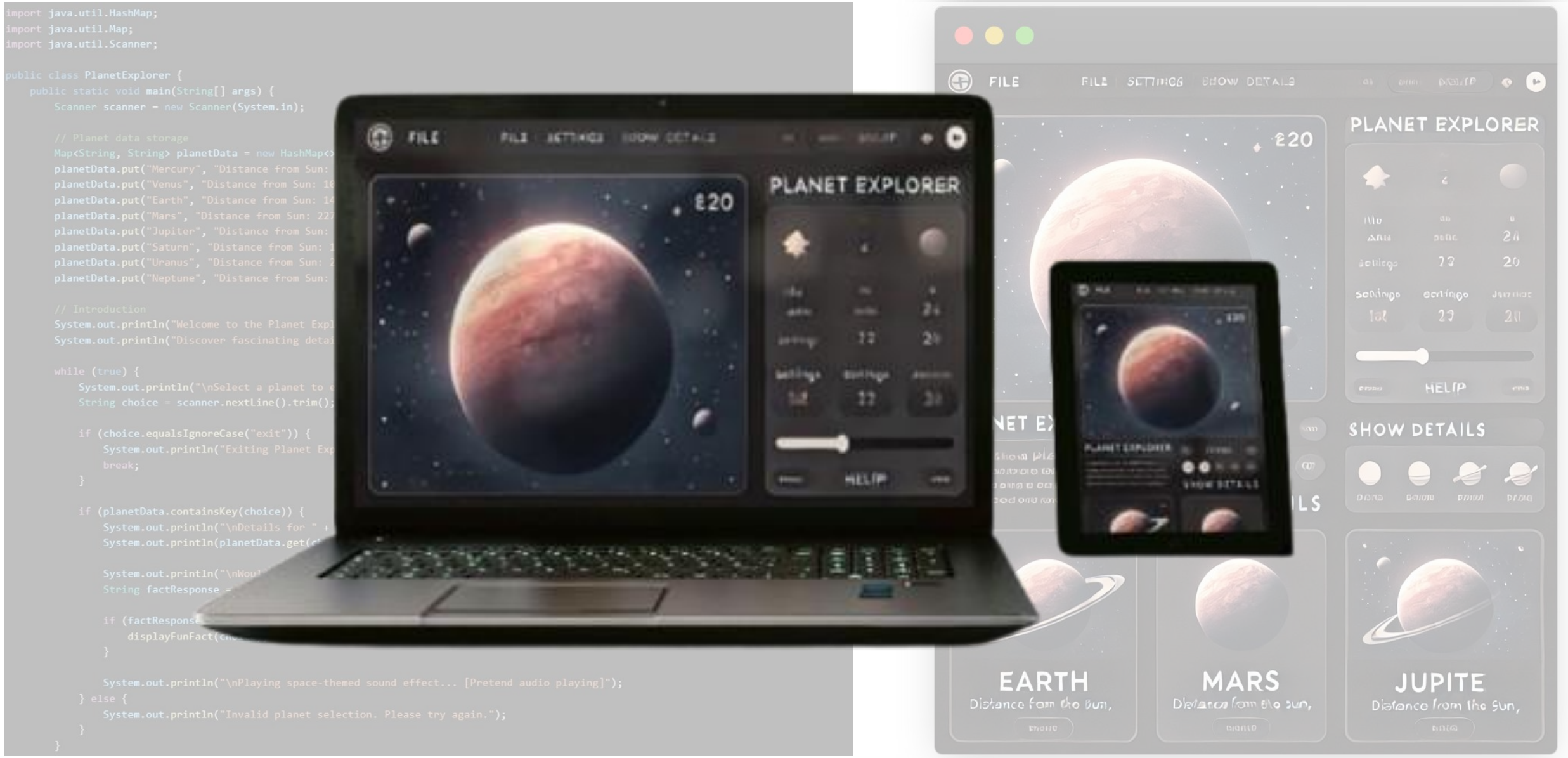
JavaFX basics

Motivations

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class PlanetExplorer {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Planet data storage
        Map<String, String> planetData = new HashMap<>();
        planetData.put("Mercury", "Distance from Sun: 57.9 million km, Gravity: 3.7 m/s², Atmosphere: Thin exosphere");
        planetData.put("Venus", "Distance from Sun: 108.2 million km, Gravity: 8.87 m/s², Atmosphere: 96% Carbon Dioxide, 3.5% Nitrogen");
        planetData.put("Earth", "Distance from Sun: 149.6 million km, Gravity: 9.8 m/s², Atmosphere: 78% Nitrogen, 21% Oxygen");
        planetData.put("Mars", "Distance from Sun: 227.9 million km, Gravity: 3.7 m/s², Atmosphere: 95% Carbon Dioxide");
        planetData.put("Jupiter", "Distance from Sun: 778.5 million km, Gravity: 24.8 m/s², Atmosphere: Hydrogen and Helium");
        planetData.put("Saturn", "Distance from Sun: 1.43 billion km, Gravity: 10.44 m/s², Atmosphere: Hydrogen and Helium");
        planetData.put("Uranus", "Distance from Sun: 2.87 billion km, Gravity: 8.69 m/s², Atmosphere: Hydrogen, Helium, and Methane");
        planetData.put("Neptune", "Distance from Sun: 4.50 billion km, Gravity: 11.15 m/s², Atmosphere: Hydrogen, Helium, and Methane");

        // Introduction
        System.out.println("Welcome to the Planet Explorer!");
        System.out.println("Discover fascinating details about different planets in our solar system.");

        while (true) {
            System.out.println("\nSelect a planet to explore (Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune) or type 'exit'
            String choice = scanner.nextLine().trim();

            if (choice.equalsIgnoreCase("exit")) {
                System.out.println("Exiting Planet Explorer. Goodbye!");
                break;
            }

            if (planetData.containsKey(choice)) {
                System.out.println("\nDetails for " + choice + ":");
                System.out.println(planetData.get(choice));

                System.out.println("\nWould you like to learn an interesting fact about " + choice + "? (yes/no)");
                String factResponse = scanner.nextLine().trim();

                if (factResponse.equalsIgnoreCase("yes")) {
                    displayFunFact(choice);
                }

                System.out.println("\nPlaying space-themed sound effect... [Pretend audio playing]");
            } else {
                System.out.println("Invalid planet selection. Please try again.");
            }
        }
```



Motivations



Objectives

- To distinguish between JavaFX, Swing, and AWT (§14.2).
- To write a simple JavaFX program and understand the relationship among stages, scenes, and nodes (§14.3).
- To create user interfaces using panes, UI controls, and shapes (§14.4).
- To use binding properties to synchronize property values (§14.5).
- To use the common properties **style** and **rotate** for nodes (§14.6).
- To create colors using the **Color** class (§14.7).
- To create fonts using the **Font** class (§14.8).
- To create images using the **Image** class and to create image views using the **ImageView** class (§14.9).
- To layout nodes using **Pane**, **StackPane**, **FlowPane**, **GridPane**, **BorderPane**, **HBox**, and **VBox** (§14.10).
- To display text using the **Text** class and create shapes using **Line**, **Circle**, **Rectangle**, **Ellipse**, **Arc**, **Polygon**, and **Polyline** (§14.11).
- To develop the reusable GUI components **ClockPane** for displaying an analog clock (§14.12).

Objectives

- To get a taste of event-driven programming (§15.1).
- To describe events, event sources, and event classes (§15.2).
- To define handler classes, register handler objects with the source object, and write the code to handle events (§15.3).
- To define handler classes using inner classes (§15.4).
- To define handler classes using anonymous inner classes (§15.5).
- To simplify event handling using lambda expressions (§15.6).
- To develop a G U I application for a loan calculator (§15.7).
- To write programs to deal with **MouseEvent**s (§15.8).
- To write programs to deal with **KeyEvent**s (§15.9).
- To create listeners for processing a value change in an observable object (§15.10).
- To use the **Animation**, **PathTransition**, **FadeTransition**, and **Timeline** classes to develop animations (§15.11).
- To develop an animation for simulating a bouncing ball (§15.12).
- To draw, color, and resize a U S map (§15.13).

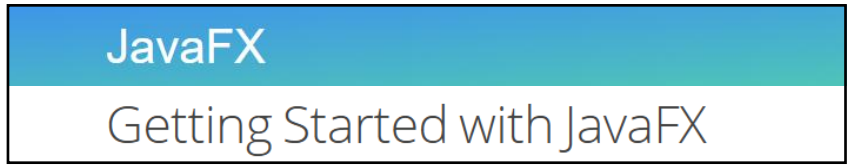
Objectives

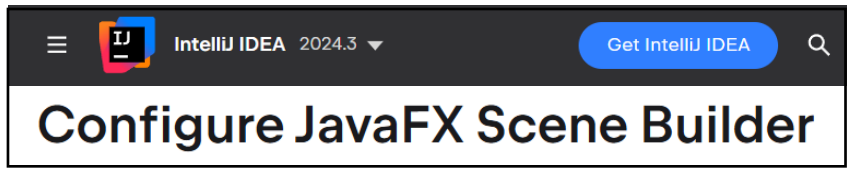
- To create graphical user interfaces with various user-interface controls (§§16.2–16.11).
- To create a label with text and graphic using the **Label** class and explore properties in the abstract **Labeled** class (§16.2).
- To create a button with text and graphic using the **Button** class and set a handler using the **setOnAction** method in the abstract **ButtonBase** class (§16.3).
- To create a check box using the **CheckBox** class (§16.4).
- To create a radio button using the **RadioButton** class and group radio buttons using a **ToggleGroup** (§16.5).
- To enter data using the **TextField** class and password using the **PasswordField** class (§16.6).
- To enter data in multiple lines using the **TextArea** class (§16.7).
- To select a single item using **ComboBox** (§16.8).
- To select a single or multiple items using **ListView** (§16.9).
- To select a range of values using **ScrollBar** (§16.10).
- To select a range of values using **Slider** and explore differences between **ScrollBar** and **Slider** (§16.11).
- To develop a tic-tac-toe game (§16.12).
- To view and play video and audio using the **Media**, **MediaPlayer**, and **MediaView** (§16.13).
- To develop a case study for showing the national flag and play anthem (§16.14).

Objectives

- To create graphical user interfaces with various user-interface controls (§§16.2–16.11).
- To specify styles for UI nodes using **JavaFX CSS** (§31.2).
- To create quadratic curve, cubic curve, and path using the **QuadCurve**, **CubicCurve**, and **Path** classes (§31.3).
- To translation, rotation, and scaling to perform coordinate transformations for nodes (§31.4).
- To define a shape's border using various types of strokes (§31.5).
- To create menus using the **Menu**, **MenuItem**, **CheckMenuItem**, and **RadioMemuItem** classes (§31.6).
- To create context menus using the **ContextMenu** class (§31.7).
- To use **SplitPane** to create adjustable horizontal and vertical panes (§31.8).
- To create tab panes using the **TabPane** control (§31.9).
- To create and display tables using the **TableView** and **TableColumn** classes (§31.10).

Install JavaFX + Scene Builder





- www.jetbrains.com/help/idea/javafx.html
- openjfx.io/openjfx-docs/
- www.jetbrains.com/help/idea/opening-fxml-files-in-javafx-scene-builder.html

JavaFX ← Swing ← AWT

GUI (**G**raphic **U**ser **I**nterface) platform

AWT

- Platform-specific and dependent on native UI components.

Swing

- More flexible than AWT but limited to **desktop** applications.

JavaFX

- **Desktop** and **mobile** applications with modern UI components.
- Requires separate installation from Java 11 onward.

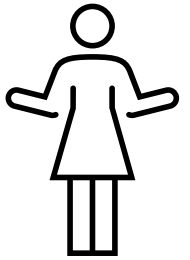
Web page: https:// www.

HTML

Hypertext Markup Language

Create the **structure**

- provides structure for the web page design
- the fundamental **building block** of web pages
- controls the layout of the content



CSS

Cascading Style Sheet

Stylize the website

- applies style to the web page elements
- targets various screen sizes to make web pages responsive
- Primarily handles the ”look and feel” of a web page

Web page: https:// www.

HTML

Hypertext Markup Language

Create the **structure**

- provides structure for the web page design
- the fundamental **building block** of web pages
- controls the layout of the content

CSS

Cascading Style Sheet

Stylize the website

- applies style to the web page elements
- targets various screen sizes to make web pages responsive
- Primarily handles the ”look and feel” of a web page

Web page: https:// www.

HTML

Hypertext Markup Language

Create the **structure**

- provides structure for the web page design
- the fundamental **building block** of web pages
- controls the layout of the content

CSS

Cascading Style Sheet

Stylize the website

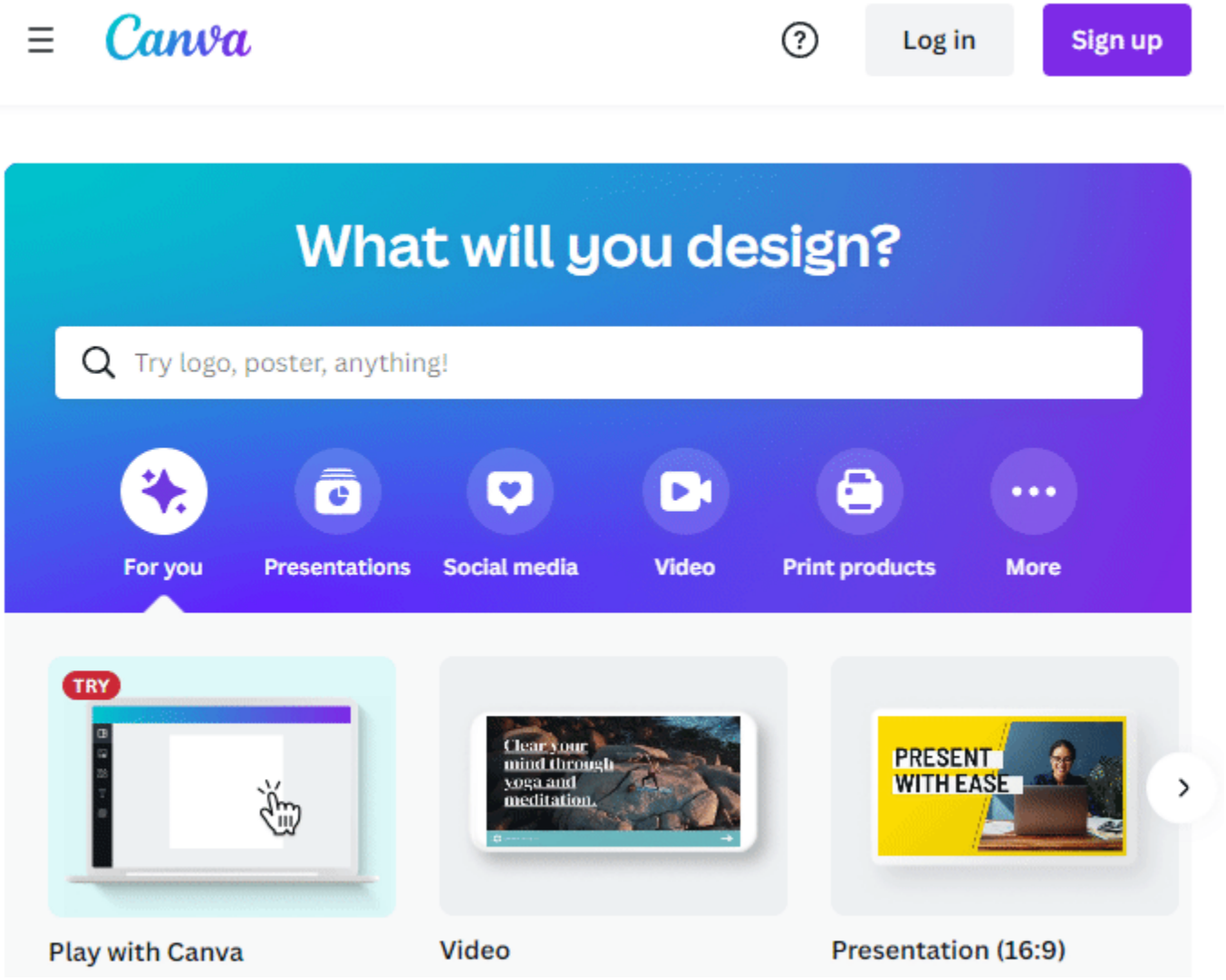
- applies style to the web page elements
- targets various screen sizes to make web pages responsive
- Primarily handles the ”look and feel” of a web page

JavaScript

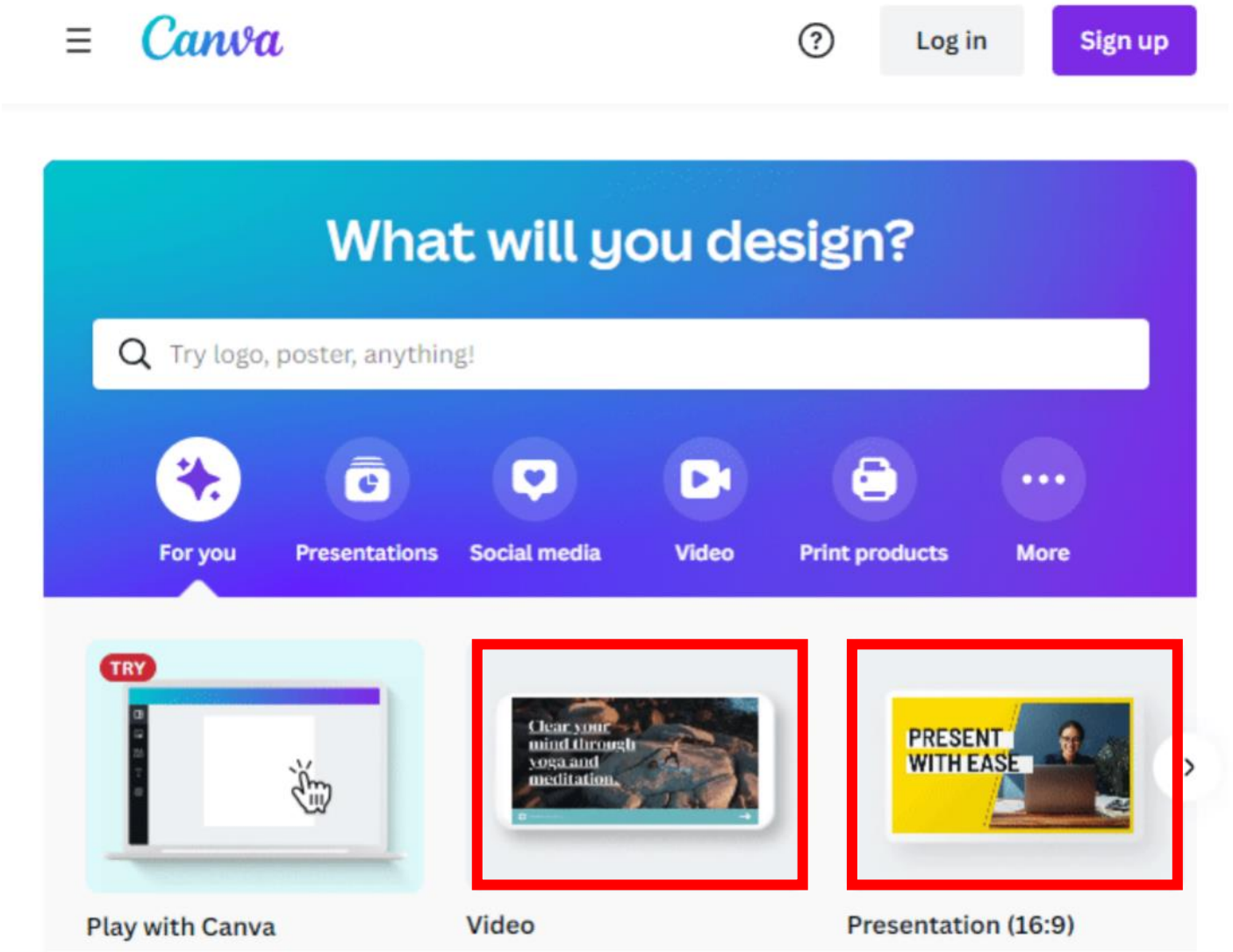
Increase **interactivity**

- adds interactivity to a web page
- handles complex functions and features
- programmatic code which enhances functionality

Web page: https:// www.



Web page: https:// www.



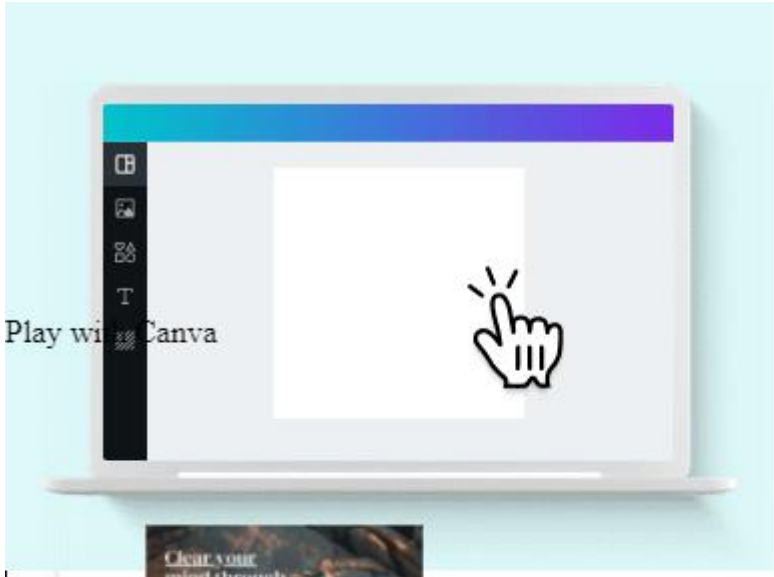
Web page: https:// www.

☰

Canva home

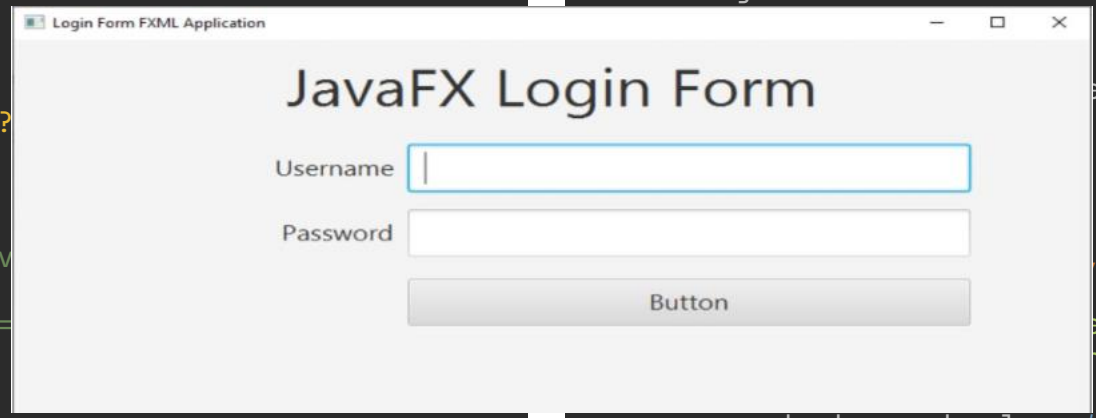
Canva

- HomeHome
- DesignDesign
  - **Social Media & Video**
    - Instagram
    - Facebook
    - Twitter
    - YouTube
    - Video Editor
  - **Marketing**
    - Business Cards
    - Flyers
    - Logos
    - Posters
    - Brochures
    - Menus



Try
(opens in a
window)
(opens in a new tab or
window)
Video

1920 × 1080 px

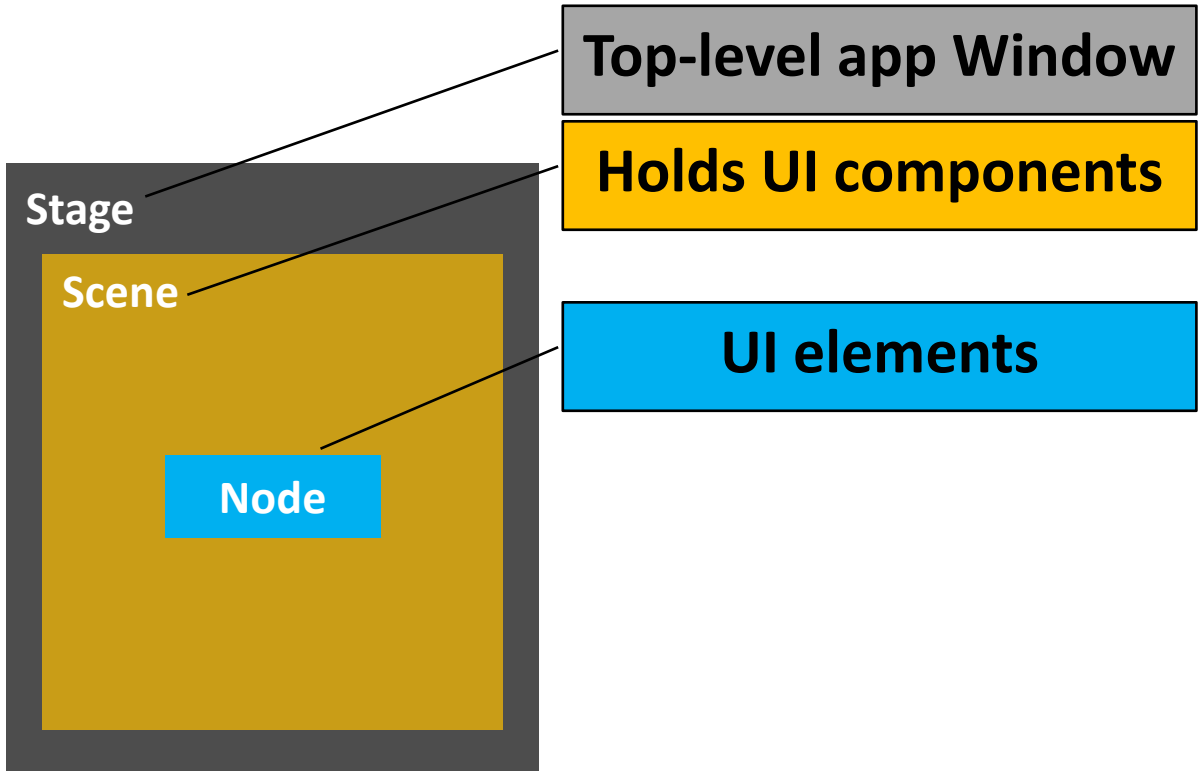



```xml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.GridPane?>

<GridPane fx:id="gridPane" xmlns="http://jav
xmlns:fx="http://javafx.com/fxml/1"
          alignment="center" hgap="10" vgap=
color: #F2F2F2;" padding="20">

    <!-- Login Form Container -->

    <!-- Title -->
    <Label text="JavaFX Login Form"
           GridPane.columnIndex="1" GridPane.rowIndex="0"
           style="-fx-font-size: 20px; -fx-font-weight: bold;"
           GridPane.columnSpan="2"/>

    <!-- Username Label and Input Field -->
    <Label text="Username" GridPane.columnIndex="0"
           GridPane.rowIndex="1"/>
    <TextField fx:id="usernameField" GridPane.columnIndex="1"
           GridPane.rowIndex="1" prefWidth="200"/>

    <!-- Password Label and Input Field -->
    <Label text="Password" GridPane.columnIndex="0"
           GridPane.rowIndex="2"/>
    <PasswordField fx:id="passwordField" GridPane.columnIndex="1"
           GridPane.rowIndex="2" prefWidth="200"/>

    <!-- Login Button -->
    <Button text="Login" fx:id="loginButton"
```

```html
<!DOCTYPE html>
<html lang="en">

ent="width=device-width, initial-

 sans-serif;

enter;
;

            background-color: #f2f2f2;
        }
    </style>
</head>
<body>
<div class="login-container">
    <h2>Login Form</h2>
    <label for="username">Username</label>
    <input type="text" id="username" placeholder="Enter Username">

    <label for="password">Password</label>
    <input type="password" id="password" placeholder="Enter Password">

    <button onclick="loginAction()">Login</button>
    <p id="message" style="color: red;"></p>
</div>

<script>
    function loginAction() {
        let username = document.getElementById("username").value.trim();
        let password = document.getElementById("password").value;
```

Essential Structure

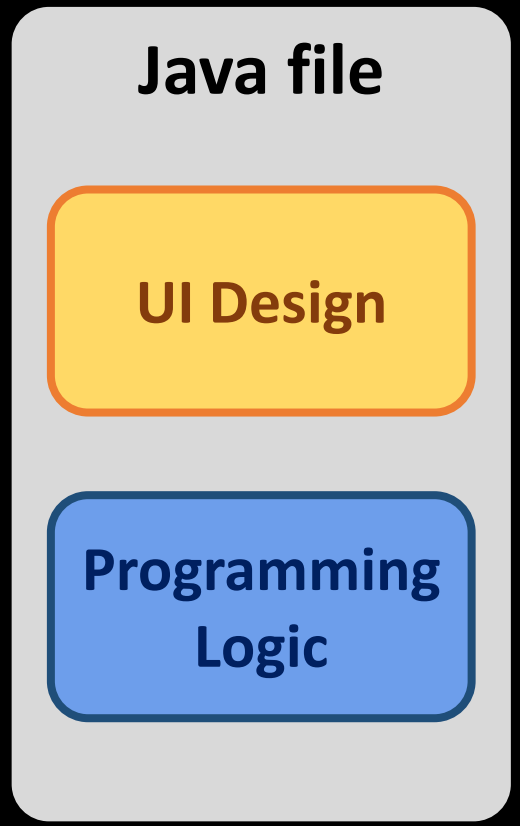
- **Application class**: Basic JavaFX class extended to manages the app's lifecycle.
- **start(Stage) method**: Entry point for building the UI
- **Stage**, **Scene**, and **Nodes**



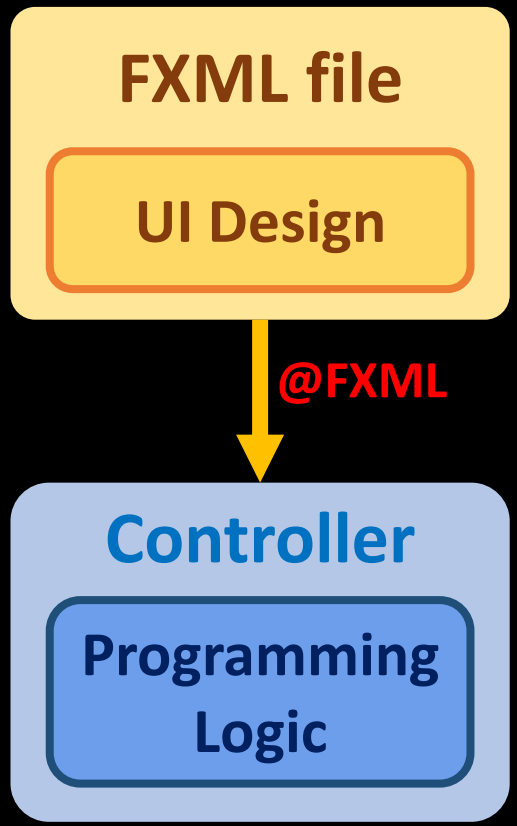

```
Stage (Top-Level Window)
└── Scene (Holds UI Elements)
    ├── Node (Base Class for All UI Components)
    │   ├── Parent (A Special Type of Node That Can Contain Children)
    │   │   ├── Pane (Generic Layout Container)
    │   │   │   ├── VBox (Vertical Layout)
    │   │   │   ├── HBox (Horizontal Layout)
    │   │   │   ├── GridPane (Grid-Based Layout)
    │   │   │   └── . . .
    │   ├── Control (UI Components - Cannot Contain Children)
    │   │   ├── Button
    │   │   ├── Label
    │   │   ├── TextField
    │   │   ├── CheckBox
    │   │   └── . . .
    │   ├── Shape (Graphical Elements)
    │   │   ├── Line
    │   │   ├── Rectangle
    │   │   └── . . .
    │   ├── ImageView (Displays Images)
```

Developing JavaFx Application

I. Single-file (Programmatic) Approach



II. FXML-based (Declarative) Approach



I. Single-file (Programmatic) Approach: .java

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {

        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");

        VBox vbox = new VBox(10, lbl, btn);

        Scene scene = new Scene(vbox, 300, 200);

        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        btn.setOnAction(e -> label.setText("Hello"));
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

UI design

Logic

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.scene.control.Button;
5  import javafx.scene.control.Label;
6  import javafx.scene.layout.VBox;
7
9
10
11
12
13
14
15
16
17
18
19
```

imports necessary JavaFX classes

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.scene.control.Button;
5  import javafx.scene.control.Label;
6  import javafx.scene.layout.VBox;

7  public class JavaFXApp extends Application {

9

10
11

12

13

14
15
16

17

18
19

}//class
```

imports necessary JavaFX classes

- **Application class** controls the **lifecycle** of a JavaFX program, handling **initialization**, **startup**, and **closing**.
- This **lifecycle** **automates** **program flow**, allowing JavaFX to manage the different stages of the **execution** without manual control.

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {









    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {









    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

- Overrides the **start method**, which serves as the **entry point** of the JavaFX application.
- Receives a **Stage object**, representing the **main application window**.

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components --------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container ------------------------------
        VBox vbox = new VBox(10, lbl, btn);

    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components --------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container ------------------------------
        VBox vbox = new VBox(10, lbl, btn);

    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

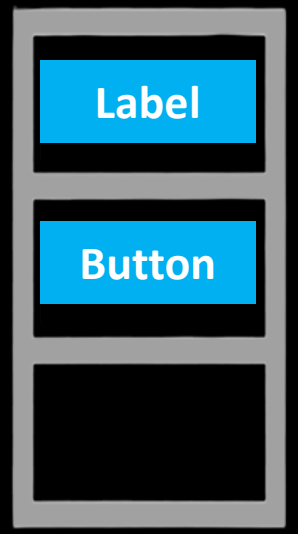
imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**



```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

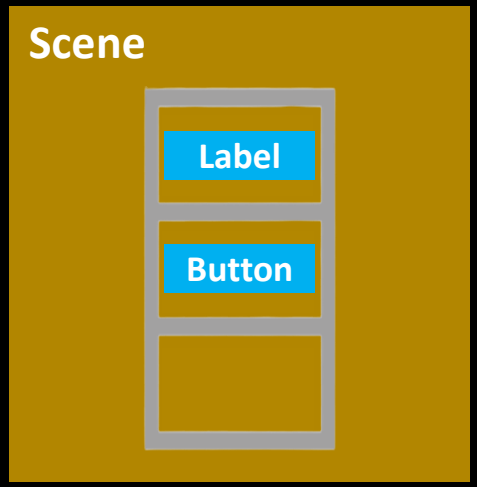
start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**

can define the window size



```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the **Stage**
set the **window title**
make the **window visible**

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

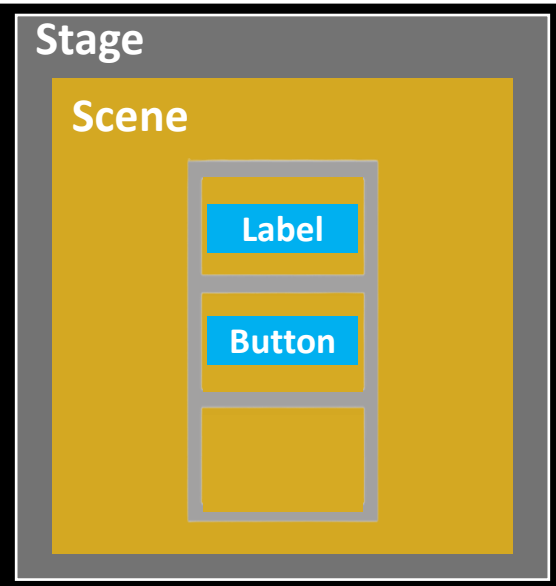
start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the **Stage**
set the **window title**
make the **window visible**



```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();



    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

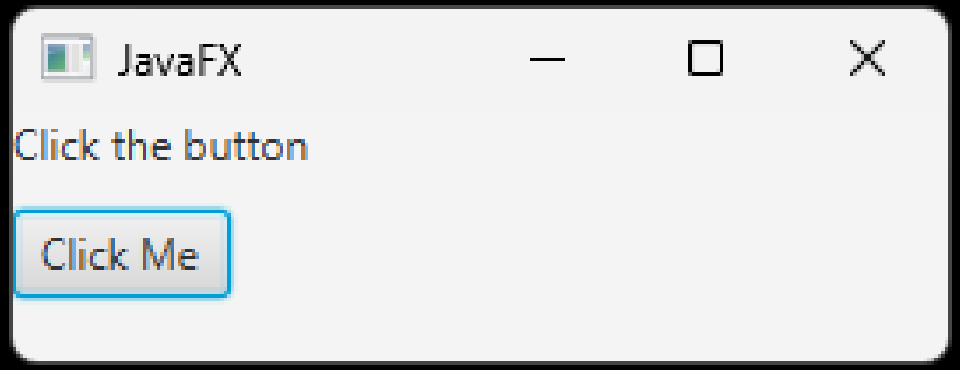
start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the Stage
set the **window title**
make the **window visible**



```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        //5.event handler -------------------------
        btn.setOnAction(e -> label.setText("Hello"));
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the **Stage**
set the **window title**
make the **window** **visible**

add **event handler** to **UI component**
Programming Logic part

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        //5.event handler -------------------------
        btn.setOnAction(e -> label.setText("Hello"));
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

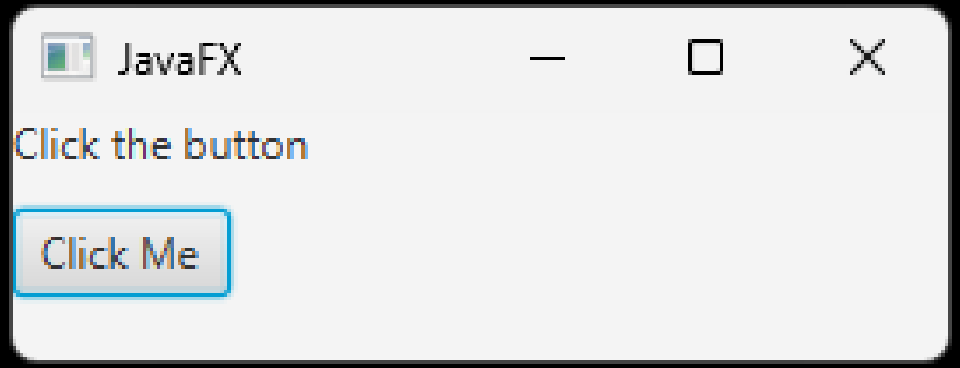
creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the **Stage**
set the **window title**
make the **window** **visible**

add **event handler** to **UI component**
Programming Logic part



```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        //5.event handler -------------------------
        btn.setOnAction(e -> label.setText("Hello"));
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

imports necessary JavaFX classes

main(): calling **launch()** invokes **start()**

start(stage**)**: entry point of a JavaFX app

creates **UI components**

container to add the **UI components**

creates a **Scene** to add the **container**
can define the window size

assigns the **Scene** to the **Stage**
set the **window title**
make the **window** **visible**

add **event handler** to **UI component**
Programming Logic part

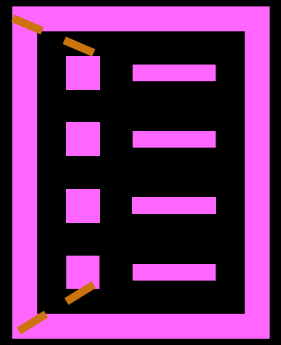
```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        //5.event handler -------------------------
        btn.setOnAction(e -> label.setText("Hello"));
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

Main.java

UI design

Logic

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;

public class JavaFXApp extends Application {
    @Override
    public void start(Stage stage) {
        //1.UI components -------------------------
        Label  lbl = new Label("Click the button");
        Button btn = new Button("Click Me");
        //2.Container -----------------------------
        VBox vbox = new VBox(10, lbl, btn);
        //3.Scene ---------------------------------
        Scene scene = new Scene(vbox, 300, 200);
        //4.Stage(Window) -------------------------
        stage.setTitle("JavaFX");
        stage.setScene(scene);
        stage.show();

        //5.event handler -------------------------
        btn.setOnAction(e -> label.setText("Hello"));
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```



JavaFX
single-file
template

II. FXML-based (Declarative) Approach: .fxml

pom.xml (FXML2) | view.fxml | Controller.java | FXMLapp.java

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>FXML2</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>FXML2</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.10.0</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
```

II. FXML-based (Declarative) Approach: .fxml

pom.xml (FXML2) | view.fxml | Controller.java | FXMLapp.java

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.geometry.*?>
4  <?import javafx.scene.control.*?>
5  <?import javafx.scene.layout.*?>
6
7  <VBox alignment="CENTER" prefHeight="170.0"
8        prefWidth="356.0" spacing="20.0"
9        xmlns:fx="http://javafx.com/fxml/1"
10       xmlns="http://javafx.com/javafx/17.0.2-ea"
11       fx:controller="org.example.fxml2.Controller">
12     <padding>
13        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
14     </padding>
15     <Label fx:id="lbl" />
16     <Button fx:id="btn" onAction="#onHelloButtonClick" text="Hello!" />
17 </VBox>
18
```

II. FXML-based (Declarative) Approach: .fxml

pom.xml (FXML2) | view.fxml | Controller.java | FXMLapp.java

```java
package org.example.fxml2;

import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class Controller {
    @FXML
    private Label lbl;

    @FXML
    protected void onHelloButtonClick() { lbl.setText("Welcome to JavaFX Application!"); }
}
```

II. FXML-based (Declarative) Approach: .fxml

pom.xml (FXML2) | view.fxml | Controller.java | FXMLapp.java

```java
package org.example.fxml2;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

public class FXMLapp extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader
                (FXMLapp.class.getResource( name: "view.fxml"));
        Scene scene = new Scene(fxmlLoader.load());
        stage.setTitle("FXML");
        stage.setScene(scene);
        stage.show();
    }

```

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
```

defines XML version and character encoding

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>
6
7
8
9
10
11
12
13
14
15
16
```

imports **UI components** & **utilities** to an **FXML** file

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7
8
9
10
11
12

13
14
15
16
    </VBox>
```

defines the top-lovel **container**,
the **root** of the UI hierarchy
where other components are placed

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8
9
10
11
12
13
14
15
16
    </VBox>
```

defines fx namespace

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8
9
10
11
12
13
14
15
16
    </VBox>
```

defines **fx namespace**

identifier for Javafx-specific attributes
(e.g., **fx**:id, **fx**:controller)
NOT a website

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8         fx:controller="Controller">

9
10
11
12

13
14
15
16
    </VBox>
```

links the **FXML** file to its **Java controller** class

the exact controller class name

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8         fx:controller="Controller">

9       <padding>
10          <Insets bottom="20.0" top="20.0"
11                  left="20.0" right="20.0" />
12      </padding>

13      <Label  fx:id="lbl"/>
14      <Button fx:id="btn"/>
15
16

    </VBox>
```

declares **UI components** & assigns **IDs** for **Java controller** access

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8         fx:controller="Controller">

9       <padding>
10          <Insets bottom="20.0" top="20.0"
11                  left="20.0" right="20.0" />
12      </padding>

13      <Label  fx:id="lbl"/>
14      <Button fx:id="btn"/>
15
16
    </VBox>
```

declares **UI components** & assigns **IDs** for **Java controller** access

links **UI components** in **FXML** to the variables (**lbl**,**btn**) in the **controller class**

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8         fx:controller="Controller">

9       <padding>
10          <Insets bottom="20.0" top="20.0"
11                  left="20.0" right="20.0" />
12      </padding>

13      <Label  fx:id="lbl"/>
14      <Button fx:id="btn"
15              text="Hello!"
16              onAction="#onHelloButtonClick"/>
    </VBox>
```

links **UI components** to a **method** in the **controller class** for **event handling**.

FXML: UI Layout

```
1   <?xml version="1.0" encoding="UTF-8"?>
2   <?import javafx.scene.control.Button?>
3   <?import javafx.scene.control.Label?>
4   <?import javafx.scene.layout.VBox?>
5   <?import javafx.geometry.Insets?>



6   <VBox alignment="CENTER" spacing="20.0"
7         xmlns:fx="http://javafx.com/fxml"
8         fx:controller="Controller">

9       <padding>
10          <Insets bottom="20.0" top="20.0"
11                  left="20.0" right="20.0" />
12      </padding>

13      <Label  fx:id="lbl"/>
14      <Button fx:id="btn"
15              text="Hello!"
16              onAction="#onHelloButtonClick"/>
    </VBox>
```

What is a Namespace?

Namespace organizes related code and **prevents naming conflicts** by allowing multiple elements (classes, functions, variables) to exist under the same name in different contexts.

It acts as a **scope** where a name remains unique within its own space, avoiding clashes with identical names in other namespaces.

A **namespace is like a folder** on your computer that contains related files, keeping them organized and preventing duplicate file names from causing issues.

What is a Namespace?

```java
package airline; // Namespace for airline-related code
public class TicketBooking { }

package train;  // Namespace for train-related code
public class TicketBooking { }
```

Even though both classes have **the same name**,
they are treated **differently**
because they belong to **separate namespaces (airline and train)**

What is a Namespace?

```java
package airline;
public class TicketBooking { }

package train;
public class TicketBooking { }
```

defines different **namespaces**

```
xmlns:math   ="http://example.com/math"
xmlns:science="http://example.com/science"

<math:   subject>Algebra</math:subject>
<science:subject>Physics</science:subject>
```

same name, but the namespaces differentiate them

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2
3
4
5
6
7
8
```

imports @FXML annotation

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Label;
3
4
5
6
7
8
```

imports **UI components**
that need to be accessed **programmatically**

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Label;
   import javafx.scene.control.Button;
3
4
5
6
7
8
```

since the button only triggers an **event** **and is not** accessed **programmatically**, it does **not** need to be **imported**

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Label;

3  public class Controller {
4
5
6
7
8

   }//Controller
```

manages **user interactions** with the FXML UI

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Label;

3  public class Controller {
4      @FXML
5
6
7
8
   }//Controller
```

-links **UI elements** from the **FXML** file to the **Java controller**,

allowing the **FXMLLoader** to inject **UI components** & recognize **event handlers**.

-must be **manually added** for any **UI components** requiring **interaction** in the controller.

Controller Class: Event Handling & Logic

```
1  import javafx.fxml.FXML;
2  import javafx.scene.control.Label;

3  public class Controller {
4     @FXML
5     private Label lbl;

6
7
8

   }//Controller
```

```
<Label  fx:id="lbl"/>
```

Controller Class: Event Handling & Logic

```java
import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class Controller {
    @FXML
    private Label lbl;

    @FXML
    protected void onHelloButtonClick() {
        lbl.setText("Welcome to JavaFX Application!");
    }
}//Controller
```

```
<Button fx:id="btn"
onAction="#onHelloButtonClick"/>
```

Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5
6
7
8
9
10
11
12
13
14
15
```

loads **FXML** files & converts them into a JavaFX **UI**

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6
7
8
9

10
11
12
13


14
15
```

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;
6
7
8
9
10
11
12
13
14
15
```

If the **FXMLLoader** fails to find or read the FXML file, an **IOException** is thrown

Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7
8
9
10
11
12
13
14
15
}//class
```

- **Application class** controls the **lifecycle** of a JavaFX program, handling **initialization**, **startup**, and **closing**.
- This **lifecycle** **automates** **program flow**, allowing JavaFX to manage the different stages of the **execution** without manual control.

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7
8
9
10
11
12
13
14     public static void main(String[] args) {
15         launch(args);
       }
   }//class
```

calling **launch()** invokes **start()**

Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9
10
11
12
13
       }//start

14     public static void main(String[] args) {
15         launch(args);
       }
   }//class
```

overrides the **start** method , which serves as the **entry point** of the JavaFX application. If UI components fails to load properly, it throws **IOException**.

Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));
10
11
12
13
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```

loads the **FXML** file, making it usable in JavaFX application
Instead of manually constructing the UI in code,
the layout is defined separately in FXML.

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

10         Scene scene = new Scene(fxmlLoader.load(), 300, 200);
11
12
13
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```

creates a **Scene** to contain **UI components**
by loading them from an **FXML** file
can also define the **window size**

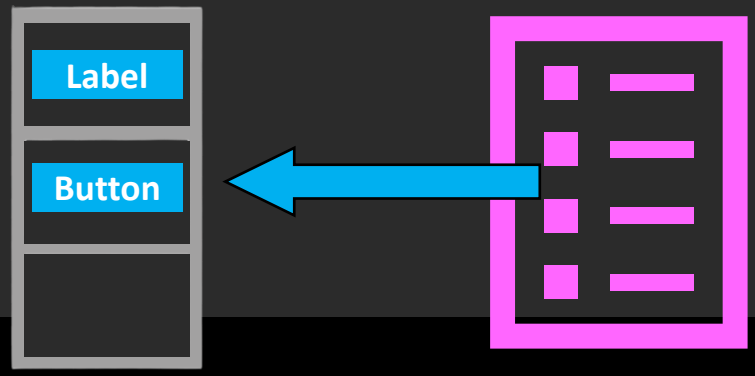
Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

10         Scene scene = new Scene(fxmlLoader.load(), 300, 200);
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```

assigns the **Scene** to the **Stage**
set the **window title**
make the **window** **visible**

Main Application Class: JavaFx Entry point

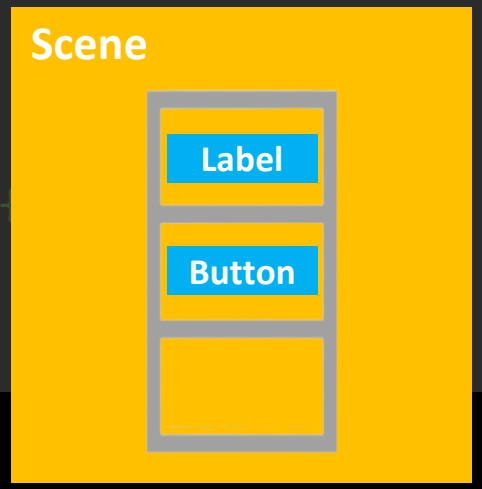
```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

10         Scene scene = new Scene(fxmlLoader.load(), 300, 200);
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```

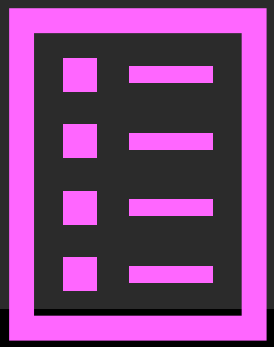


FXML file

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

10         Scene scene = new Scene(fxmlLoader.load(), 300, 200);
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```

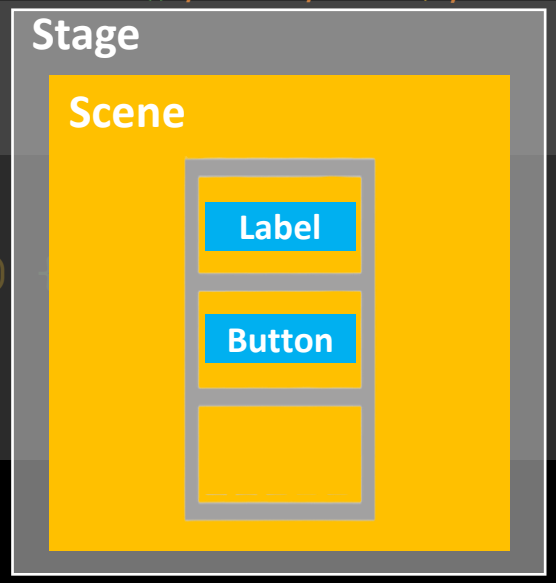


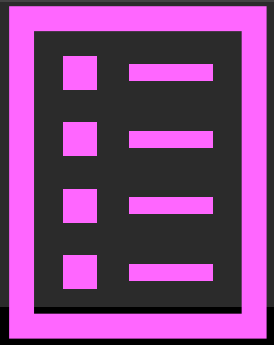


FXML file

Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

10         Scene scene = new Scene(fxmlLoader.load(), 300, 200);
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args) {
15         launch(args);
   }
}//class
```





FXML file

Main Application Class: JavaFx Entry point

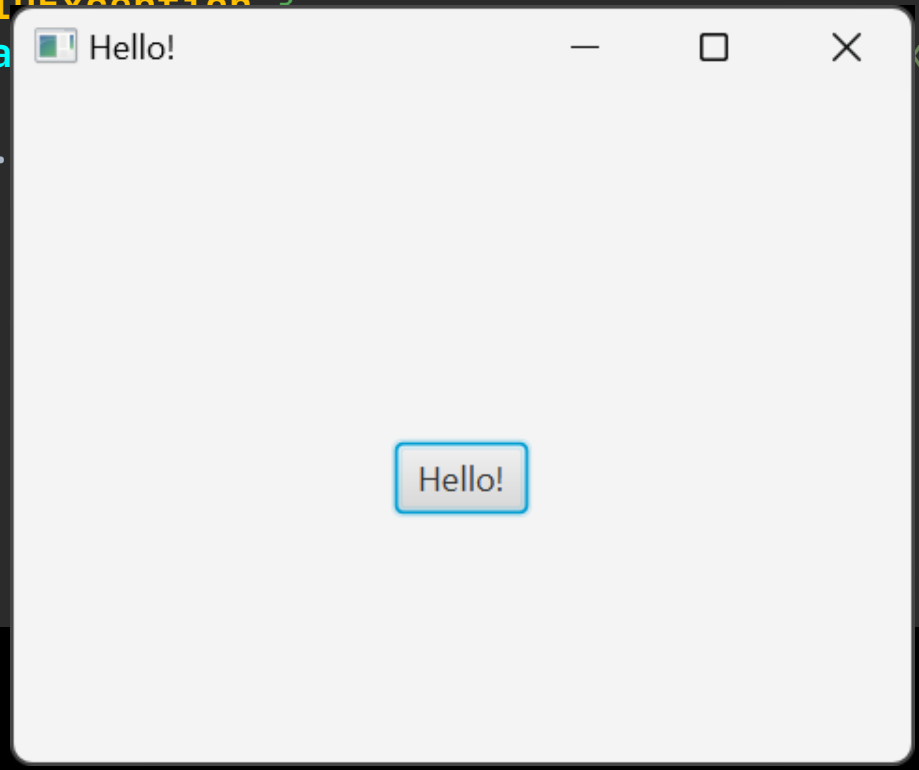
```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;
import javafx.fxml.FXMLLoader;
import java.io.IOException;

public class FXMLapp extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(FXMLapp.class.getResource("view.fxml"));

        Scene scene = new Scene(fxmlLoader.load(), 300, 200);
        stage.setTitle("Hello!");
        stage.setScene(scene);
        stage.show();
    }//start

    public static void main(String[] args) {
        launch(args);
    }
}//class
```

Ctrl + Shift + F10

^ ⇧ R,

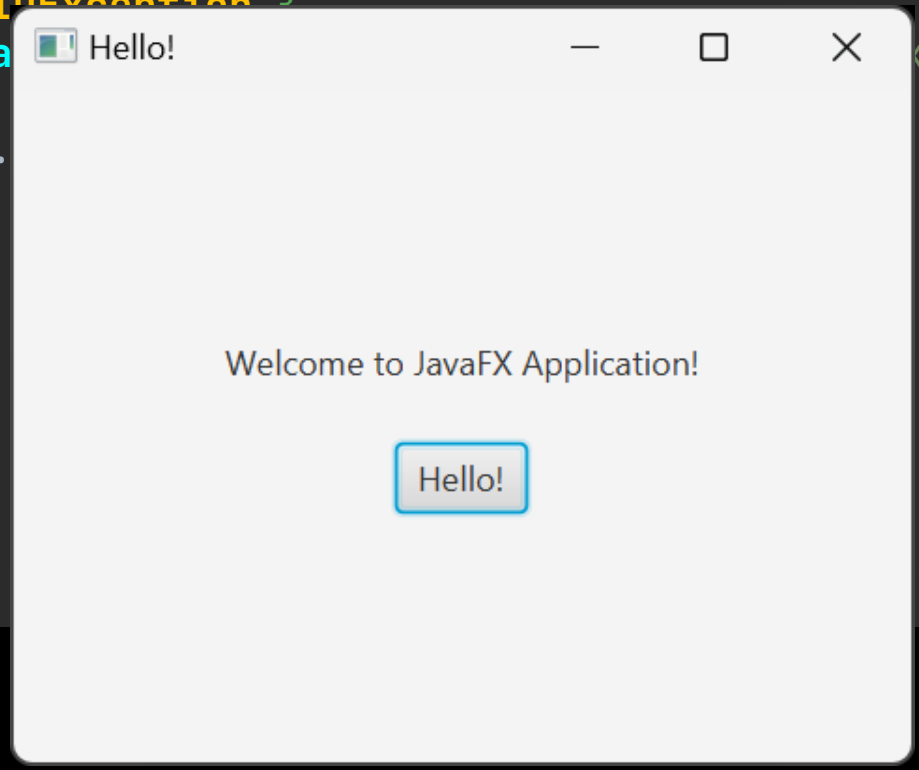
Main Application Class: JavaFx Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoa                    xml"));

10         Scene scene = new Scene(fxmlLoader.
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args)
15         launch(args);
   }
}//class
```



Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoa                    xml"));

10         Scene scene = new Scene(fxmlLoader.
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args)
15         launch(args);
   }
}//class
```



Main Application Class: JavaFX Entry point

```
1  import javafx.application.Application;
2  import javafx.scene.Scene;
3  import javafx.stage.Stage;
4  import javafx.fxml.FXMLLoader;
5  import java.io.IOException;

6  public class FXMLapp extends Application {
7      @Override
8      public void start(Stage stage) throws IOException {
9          FXMLLoader fxmlLoader = new FXMLLoa                    xml"));

10         Scene scene = new Scene(fxmlLoader.
11         stage.setTitle("Hello!");
12         stage.setScene(scene);
13         stage.show();
       }//start

14     public static void main(String[] args)
15         launch(args);
   }
}//class
```